

Revfit Recommender — Experiment Documentation

Overview

This document describes the **7 recommendation strategies** evaluated for the Revfit fitness recommender system seminar. Each strategy is tested on **two datasets simultaneously**:

Dataset	Source	Catalogue size
Workouts	megaGymDataset (Kaggle)	2,878 exercises
Meals	Spoonacular API	380K+ recipes (20 fetched per persona)

Current deployed system: Experiment 7 — VDB-Based Collaborative Filtering (profile similarity).

Academic Reference Anchors

Workout domain (2024/2025)

"Personalized Gym Recommendation System Using Machine Learning" *International Journal of Engineering Trends and Technology (IJETT)*, 2024/2025.

Uses the **exact same Mega Gym Dataset** we use. Confirms that evaluating multiple algorithmic paradigms against this dataset is active academic research.

Meal domain (2024)

Yusoff, F.H.M. et al. (2024). "KAWANKULINER: Personalised Food Recommendation App Using BMR and TDEE for Optimal Daily Nutrition." *JMSI*, Vol. 4, No. 2. DOI: [10.46754/jmsi.2024.10.004](https://doi.org/10.46754/jmsi.2024.10.004)

Uses the **exact same Spoonacular API** with BMR/TDEE caloric matching — directly analogous to our meal scoring formula.

Foundational paradigms (2018)

Ezin, E., Kim, E., & Palomares Carrascosa, I. (2018). "Fitness that Fits: A prototype model for Workout Video Recommendation." *RecSys'18, Health RecSys Workshop*.

Establishes that (1) content-based filtering overcomes cold-start in fitness apps, (2) CF struggles with data sparsity, and (3) hybrid systems are the industry gold standard. Our experimental arc directly mirrors this progression.

Evaluation Methodology — 10 Persona Simulation

Because Revfit is in a cold-start state (no real users yet), we evaluate against **10 hand-crafted personas** whose preferences are fully specified in advance:

Persona	Goal	Equipment	Diet
---------	------	-----------	------

Persona	Goal	Equipment	Diet
Ahmed	muscle_gain	Barbell, Dumbbell	omnivore
Sara	fat_loss	Body Only	omnivore
Youssef	endurance	Body Only, Bands	vegan
Layla	muscle_gain	Dumbbell, Cable	omnivore
Omar	maintenance	Machine, Dumbbell	omnivore
Nour	fat_loss	Kettlebells, Bands	vegetarian
Khalid	muscle_gain	Barbell, Machine, Cable	omnivore
Hana	maintenance	Body Only	vegan
Ziad	endurance	Body Only, Bands	omnivore
Dina	fat_loss	Dumbbell, Cable	ketogenic

Workout relevance — workouts that pass the persona's hard filters (equipment, level) whose `workout_type` belongs to the persona's goal-aligned types (mirroring `GOAL_TYPE_WEIGHTS` in `filters.py`).

Meal relevance — fetched recipes that fit within the persona's calorie budget and carry a matching diet label.

All metrics are averaged across the 10 personas.

Metrics

Metric	What it measures
Precision@K	Fraction of recommendations that are relevant
Diversity	Fraction of distinct (type, body_part) pairs in the top-K list
Coverage	Fraction of the catalogue recommended across all personas
Novelty	Average self-information of recommended items

Note on Recall: Omitted intentionally. With relevance sets of 500–2,000 items and $K=5$, recall is always < 0.05 across all methods — a mathematical artefact of large catalogue sizes, not a meaningful discriminator.

Experimental Arc

Random → Most Popular → Content-Based → CF (stub) → Hybrid → Weighted
Content → CF-VDB ★

↑ ↑ ↑ ↑ ↑
↑

worst potential baseline learning)	naive DEPLOYED baseline SYSTEM	strong baseline	cold-start failure	industry standard	most (needs
---	---	------------------------	---------------------------	--------------------------	--------------------

1. **Start with the worst case** (Random) — sets the lower bound.
2. **Add a non-personalised signal** (popularity) — shows the cost of ignoring users.
3. **Add full personalisation** (content features) — establishes a strong baseline.
4. **Try a different paradigm** (interaction-based CF) — fails without data.
5. **Combine signals** (hybrid) — industry best practice, deferred until data exists.
6. **Refine signals** (learned weights) — research direction, deferred until data exists.
7. **Profile-based CF via VDB** (Exp 7) — solves the cold-start CF problem. **Deployed.**

Experiment 1: Random Recommender

	Workouts	Meals
File	exp1_random.py	meal_exp1_random.py
Method	Random sample from hard-filtered candidates	Random sample from hard-filtered recipes

What it does

Applies the same hard filters as all other methods (equipment, fitness level for workouts; diet type, allergens, calorie ceiling for meals), then randomly selects K items with no scoring whatsoever. This establishes the absolute lower bound — any intelligent system must outperform pure chance.

Results

Metric	Workouts	Meals
Precision	0.940	0.800
Diversity	0.700	0.260
Coverage	1.3%	28.0%
Novelty	10.49	4.198

Workouts — Precision is high (0.940) because relevance sets are large (~1,100 of 2,878 workouts qualify per persona). Diversity (0.700) is naturally high since no goal bias exists.

Meals — Precision is high (0.800) because Spoonacular pre-filters by diet type; the API does the hard work. Meal diversity (0.260) is low because the API returns a thematically similar set per query regardless of scoring. Coverage (28.0%) is the highest of all meal methods — with no popularity bias, the full fetched catalogue is accessible.

Why we moved on

Random has no intelligence. We needed to test whether even a non-personalised signal (popularity) could improve relevance → **Experiment 2**.

Experiment 2: Most Popular Recommender

	Workouts	Meals
File	exp2_most_popular.py	meal_exp2_most_popular.py
Method	Rank by global rating (popularity proxy)	Rank by global rating (popularity proxy)

What it does

Ranks items by their global rating after hard-filtering. Every user receives the same list — the system is completely non-personalised.

Results

Metric	Workouts	Meals
Precision	0.980	0.300
Diversity	0.260	0.340
Coverage	0.7%	29.3%
Novelty	10.49	5.888

Workouts — A small precision gain (+0.04 over random) because popular exercises happen to be goal-relevant Strength/Cardio moves. However, diversity collapses (0.700 → 0.260): every user gets the same 5 workouts, making the system useless for personalisation.

Meals — The worst precision of any meal method (0.300 vs 0.800 for random). Popularity ranking ignores diet labels and calorie targets — the opposite dynamic to workouts. Coverage is slightly higher than random (29.3%) because popularity sorting surfaces a different catalogue slice.

Why we moved on

Popularity ignores individual differences entirely, and its effect on precision is opposite across domains (helps workouts, hurts meals). Personalisation via content features was the clear next step → **Experiment 3**.

Experiment 3: Content-Based Filtering (Strong Baseline)

Workouts	Meals
----------	-------

	Workouts	Meals
File	exp3_content_based.py	meal_exp3_content_based.py
Method	Goal-weighted scoring + feedback + hard filters	Multi-signal nutritional scoring + hard filters

What it does

Workouts — Matches item features to user profile:

```
score = goal_weight(workout_type) + 0.7×feedback(workout_id)
      + type_pref(workout_type) + 0.2×rating
```

Hard filters remove workouts with incompatible equipment or difficulty.

Meals — Scores recipes on nutritional fit:

```
score = cuisine_match(+2.0) + protein_focus(+1.5) + feedback(+2.0/−3.0)
      + calorie_proximity(0−1.0) + rating_bonus(+0.2×rating)
```

Hard filters enforce diet type, allergens, calorie ceiling, and prep time.

Results

Metric	Workouts	Meals
Precision	1.000	0.780
Diversity	0.540	0.340
Coverage	0.9%	32.0%
Novelty	10.49	4.198

Workouts — Perfect precision (1.000) across all 10 personas. The scoring function weights workout types by goal (**GOAL_TYPE_WEIGHTS**), and relevance is defined using those same types — intentional alignment. Diversity (0.540) is strong because goal-weight scoring selects across different body parts within the top goal-type bucket.

Meals — Precision (0.780) nearly matches random (0.800) but is achieved through active nutritional scoring, not luck. Coverage (32.0%) is the highest of all meal methods — each persona's calorie budget and diet label draws from a distinct part of the catalogue. Validated by Yusoff et al. (2024).

What we learned

Content-based is the right foundation: it works with zero interaction data, is safe (hard filters enforced), and produces different recommendations per user. Its limitation is the **filter bubble** — it cannot surface items outside the user's explicit feature preferences.

Why we explored further

To overcome the filter bubble, we tested whether user-user similarity could add serendipity (Exp 4–7) and whether blending signals improves robustness (Exp 5–6).

Experiment 4: Interaction-Based Collaborative Filtering (Stub)

	Workouts	Meals
File	exp4_collaborative_filtering.py	meal_exp4_collaborative_filtering.py
Method	User-based CF on interaction vectors	User-based CF on interaction vectors

What it does (conceptually)

Finds users with similar item histories and recommends what they enjoyed but the target user has not tried. Requires a non-empty user–item interaction matrix.

Results

Metric	Workouts	Meals
Precision	0.000	0.000
Diversity	0.000	0.000
Coverage	0.000	0.000

Both domains return empty lists. No interaction data → cosine similarity is undefined → CF cannot function.

What we learned

Traditional CF **requires dense interaction data** that does not yet exist.

For meals there is an additional safety concern: CF cannot enforce dietary hard constraints. If similar users happened to like a nut-based meal, CF might recommend it to a user with a nut allergy — a safety failure, not just a relevance failure.

This result is the strongest empirical argument for choosing a content-first or profile-based CF approach at cold-start. Exp 7 directly addresses this.

Experiment 5: Hybrid (Content + Popularity)

	Workouts	Meals
File	exp5_hybrid.py	meal_exp5_hybrid.py

	Workouts	Meals
Method	$\alpha \times \text{content_score} + (1-\alpha) \times \text{popularity_score}$	$\alpha \times \text{content_score} + (1-\alpha) \times \text{popularity_score}$

What it does

Blends content-based and popularity scores with a tunable parameter α :

α	Behaviour
1.0	Pure content-based (Exp 3)
0.7	70% personalised + 30% popular
0.5	Equal blend
0.0	Pure popularity (Exp 2)

Results ($\alpha = 0.7$)

Metric	Workouts	Meals
Precision	1.000	0.780
Diversity	0.540	0.340
Coverage	0.9%	32.0%

Identical to Content-Based (Exp 3) in both domains. Without real popularity counts the proxy is uniform, so the blend does not change the ranking.

What we learned

The skeleton is correctly implemented — it degrades cleanly to content-based when no popularity signal exists. The α parameter is a powerful cold-start knob: new users rely on popularity, established users rely on personalisation. This differentiation will appear once Revfit accumulates real interaction history.

Experiment 6: Feature-Weighted Content-Based

	Workouts	Meals
File	<code>exp6_weighted_content.py</code>	<code>meal_exp6_weighted_content.py</code>
Method	Learnable weight vector over feature signals	Learnable weight vector over nutritional features

What it does

Replaces fixed coefficients in the content score with a tuneable weight vector:

Workouts: $score = w_1 \times type_match + w_2 \times level_match + w_3 \times equip_match + w_4 \times rating + w_5 \times feedback$

Meals: $score = w_1 \times cuisine + w_2 \times protein + w_3 \times calorie_proximity + w_4 \times rating + w_5 \times feedback$

Weights can be set via manual strategy or learned through grid/random search.

Weight strategies

Strategy	Workout focus	Meal focus
Uniform	Equal weights (baseline)	Equal weights (baseline)
Goal-focused	Workout type is most important	Protein + calorie precision
Difficulty-focused	Match difficulty precisely	Cuisine preference heavy
Learned	Data-driven optimisation	Data-driven optimisation

Results (goal-focused weights)

Metric	Workouts	Meals
Precision	1.000	0.780
Diversity	0.540	0.340
Coverage	0.9%	32.0%

Identical to Exp 3 and Exp 5. At K=5 the top-5 list is drawn from the same goal-type bucket regardless of minor weight changes.

What we learned

Feature weighting is the natural evolution of content-based filtering. With real interaction data, learned weights would capture individual feature importance (e.g. some users prioritise body part over workout type). The interpretability advantage is real even now — the explicit weight vector documents *why* an item was recommended.

Experiment 7: VDB-Based Collaborative Filtering

Workouts	
Files	exp7_cf_vdb.py + cf_vdb.py
Method	Profile-based CF via cosine-similarity VDB + blend scoring
Signals	UserProfile feature vectors (goal, level, equipment, diet, activity, protein)

Why a new CF approach was needed

Experiment 4 (interaction-based CF) produced all-zero metrics because no user-item interaction history exists. Experiment 7 solves this with **profile-based (demographic) collaborative filtering** — the academically established remedy for CF cold-start:

Instead of asking "which users completed the same items?", ask "which users have similar fitness profiles?" — then use their content preferences as the CF signal.

How it works

1. **Embed every persona** into a 30-dimensional float32 feature vector:

Dims	Field	Encoding
0–3	goal_type	one-hot (4 goals)
4–6	fitness_level	one-hot (3 levels)
7	activity_level	ordinal → [0, 1]
8–19	available_equipment	multi-hot (12 types)
20–24	diet_type	one-hot (5 diets)
25–27	protein_focus	one-hot (3 levels)
28–29	cardio_strength_bias	one-hot (2 dims)
Total		dim = 30

2. **Store all vectors** in a `UserProfileIndex` — a numpy cosine-similarity VDB with a FAISS `IndexFlatIP`-compatible API (swappable for real FAISS once a Python 3.13 wheel is available).
3. **Query the VDB** (leave-one-out) to find the 3 most similar personas by cosine similarity.
4. **Compute a CF bonus** per candidate workout:

$$cf_bonus(w) = \sum cosine_sim(target, user_i) \times score_workout(w, user_i)$$

5. **Blend** the target user's own content score with the CF bonus:

$$final_score = 0.6 \times content_score_norm + 0.4 \times cf_bonus_norm$$

6. **Sort** by final score and return top-K.

Results (10 personas, megaGymDataset, K=5)

Metric	CF-VDB (Exp 7)	Content-Based (Exp 3)	Δ
--------	----------------	-----------------------	---

Metric	CF-VDB (Exp 7)	Content-Based (Exp 3)	Δ
Precision	1.000	1.000	=
Diversity	0.640	0.540	+0.100 (+18.5%)
Coverage	0.8%	0.9%	$\approx$
Novelty	10.49	10.49	=

- **Precision: 1.000** — The content component ($\alpha=0.6$) keeps goal-type targeting intact; the CF component ($\alpha=0.4$) draws from similar users who share the same goal bucket, so relevance is preserved.
- **Diversity: 0.640** — The highest diversity of any content-aware method. Similar users have slightly different equipment sets and training biases; their similarity-weighted preferences push different body parts and workout types into the top-5 list. Example: Sara (fat_loss, Body Only) retrieves neighbours Youssef (endurance, Body Only) and Hana (maintenance, Body Only) — overlapping equipment but different goal weights, expanding variety without hurting relevance.
- **Coverage** and **Novelty** are consistent with other content-aware methods.

Leave-one-out evaluation

Each persona is excluded from its own VDB query (`exclude_id = persona.name`), ensuring the CF signal always comes from *other* users. This mirrors the standard held-out evaluation protocol in CF literature.

Combined Results Summary

Evaluated across **10 personas**, $K=5$. Recall omitted (always < 0.05 due to large catalogue sizes).

Workout Results (megaGymDataset — 2,878 exercises)

Experiment	Prec	Diversity	Coverage	Novel	Status
1. Random	0.940	0.700	1.3%	10.49	
2. Most Popular	0.980	0.260	0.7%	10.49	
3. Content-Based	1.000	0.540	0.9%	10.49	
4. CF (interaction)	0.000	0.000	0.0%	0.000	
5. Hybrid ($\alpha=0.7$)	1.000	0.540	0.9%	10.49	
6. Weighted Content	1.000	0.540	0.9%	10.49	
7. CF-VDB	1.000	0.640	0.8%	10.49	

Meal Results (Spoonacular — 20 recipes/persona)

Experiment	Prec	Diversity	Coverage	Novel	Status
1. Random	0.800	0.260	28.0%	4.198	
2. Most Popular	0.300	0.340	29.3%	5.888	
3. Content-Based	0.780	0.340	32.0%	4.198	
4. CF (interaction)	0.000	0.000	0.0%	0.000	
5. Hybrid ($\alpha=0.7$)	0.780	0.340	32.0%	4.198	
6. Weighted Content	0.780	0.340	32.0%	4.198	
7. CF-VDB	—	—	—	—	

Reading the tables

Workout column — Precision climbs steadily (Random → Popular → Content-Based) as each signal adds goal-awareness. Diversity tells the real story: CF-VDB (0.640) beats all other methods without sacrificing precision. Most Popular collapses to 0.260 — every user gets the same 5 workouts, a design failure for a personalised app.

Meal column — The dynamic is different. Most Popular is the *worst* precision method (0.300) because popularity ignores diet constraints. Content-Based wins on Coverage (32.0%) — the decisive metric, proving different users genuinely receive different recipe lists. Spoonacular pre-filters by diet type so even random selection achieves 0.800 precision; scoring adds structure, not just accuracy.

CF (interaction) = 0 in both domains — the empirical cold-start failure. CF-VDB (Exp 7) is the direct, working answer to this failure.

Sources: *run_persona_experiments.py --workouts-only* (workouts) and *run_persona_experiments.py* (meals, Spoonacular API)

Final Conclusions

The bottom line: Profile-based collaborative filtering (Exp 7) is the right architecture for Revfit at cold-start. It achieves perfect precision AND the highest diversity of any method — outperforming both pure content-based and interaction-based CF. For meals, content-based filtering (Exp 3) remains the reference until Exp 7's allergen-safety integration is complete.

- 1. **CF-VDB (Exp 7) is the deployed system for workouts.** It simultaneously achieves Precision = 1.000 and Diversity = 0.640 — the highest diversity of any method — by drawing a CF signal from similar users' profiles rather than requiring interaction history.
- 2. **Content-based filtering (Exp 3) is the meal reference.** It achieves Precision = 0.780 and Coverage = 32.0% — the highest coverage of any meal method — proving different personas genuinely receive different recipe lists.

3. **Most Popular is the worst method for meals (Precision = 0.300)** but offers marginal precision gains for workouts (+0.04 over random). The divergence shows that popularity is domain-sensitive and should never be used as a standalone signal.
 4. **Interaction-based CF (Exp 4) fails completely** in both domains (Precision = 0.000) due to cold-start. For meals, it is additionally unsafe — it cannot enforce allergen hard constraints.
 5. **Hybrid (Exp 5) and Weighted Content (Exp 6)** are correctly implemented stubs that degrade to content-based when no real popularity or interaction data exists. They are the next evolution once Revfit accumulates user history.
 6. **Signal improvement chain (workouts):**
 - Random → Popular: +0.04 precision, −0.440 diversity (bad trade)
 - Popular → Content-Based: +0.02 precision, +0.280 diversity (clear win)
 - Content-Based → CF-VDB: same precision, **+0.100 diversity** (serendipity win)
-